Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Program: Software Engineering

Operating Systems

Laboratory Work No. 9

Watching System Calls with strace

Track A: the real OS, from the outside (Linux)

Elaborated by:
st. gr. FAF-241
Ermicev Sofia

Verified by:
Patricia Reițman

Chișinău – 2026

Contents

1 Introduction **2**

2 Part 1. See the Calls of a Real Command **2**

2.1 Observe 3

3 Part 2. A Program That Calls the Kernel Directly **3**

3.1 Observe 4

4 Part 3. Count One Kind of Call **5**

5 Conclusion **5**

1. Introduction

The goal of this laboratory work was to observe the boundary between a Linux user-space program and the kernel using `strace`. I traced the system calls of an ordinary `ls -l` command, implemented a C program using `open()`, `write()`, `read()` and `close()`, and counted the resulting `openat()` calls.

All tests were performed on Ubuntu under WSL2 as user `sofia`, in `/home/sofia/os`. The installed `strace` version was 6.8. The terminal outputs reproduced below are from the actual executions performed for this laboratory; the complete traces were also saved as text files.

```
$ whoami
sofia
$ pwd
/home/sofia/os-lab9
$ strace --version
strace -- version 6.8
```

Figure 1: Environment check and strace version

2. Part 1. See the Calls of a Real Command

I ran `ls -l` under `strace`, restricting the displayed events to `openat`, `read`, `write`, and `close`. The `-f` option also requests tracing of any child processes.

```
$ strace -f -e trace=openat,read,write,close ls -l
```

Figure 2: Command used to inspect the system calls of ls

The trace included attempts to open shared libraries, locale files, `/proc/filesyst` and the current directory. An `ENOENT` result on a locale lookup means that a candidate path was not found; this did not stop `ls` from running. At the time of the first run, the working directory was empty.

```
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
read(3, "nodev\tsysfs\nnodev\ttmpfs\nnodev\tbd"..., 1024) = 478
read(3, "", 1024)                       = 0
close(3)                                = 0
openat(AT_FDCWD, ".", O_RDONLY|O_NONBLOCK|O_CLOEXEC|O_DIRECTORY) = 3
close(3)                                = 0
write(1, "total 0\n", 8)                = 8
close(1)                                = 0
close(2)                                = 0
+++ exited with 0 +++
```

Figure 3: Selected lines from the real ls trace

2.1. Observe

Which system call does `ls` use to print to the screen? To which descriptor does it write?

The output was printed using `write()`. The trace contains `write(1, "total 0\n", 8) = 8`: descriptor `1` is standard output (`stdout`), and eight bytes were written successfully. Even a simple directory listing uses many system calls for initialization and accessing files, not only for printing the result.

3. Part 2. A Program That Calls the Kernel Directly

I created `syscalls.c` to open `note.txt`, write `hello, kernel` followed by a newline, close the file, reopen it read-only, and read the bytes back. The program uses the file-descriptor system-call interface rather than `fprintf` or `fread` for file contents.

```
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <stdio.h>

int main(void) {
    const char *msg = "hello, kernel\n";

    int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    printf("open returned descriptor %d\n", fd);

    ssize_t n = write(fd, msg, strlen(msg));
    printf("write sent %zd bytes\n", n);
    close(fd);

    fd = open("note.txt", O_RDONLY);
    char buf[64];
```

```
    n = read(fd, buf, sizeof(buf) - 1);
    buf[n] = '\0';

    printf("read got %zd bytes: %s", n, buf);
    close(fd);

    return 0;
}
```

Figure 4: Implementation of syscalls.c

```
$ gcc -Wall -Wextra -o syscalls syscalls.c
$ ./syscalls
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
$ cat note.txt
hello, kernel
```

Figure 5: Compilation and actual program output

The file contained the expected 14 bytes, including the final newline. I then traced the program's system calls.

```
$ strace -e trace=openat,write,read,close ./syscalls
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(3, "hello, kernel\n", 14)         = 14
close(3)                                = 0
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
read(3, "hello, kernel\n", 63)          = 14
close(3)                                = 0
+++ exited with 0 +++
```

Figure 6: Relevant events from the real syscalls trace

The trace also contained library initialization and `write(1, ...)` calls corresponding to the program's `printf()` messages. The displayed file operations show that Linux used `openat()` to implement the C `open()` requests.

3.1. Observe

Why did the first `openat()` for `note.txt` return descriptor 3 rather than 0, 1, or 2?

A process normally starts with the three standard descriptors already open: `0` for standard input (`stdin`), `1` for standard output (`stdout`), and `2` for standard error (`stderr`). Therefore the first free descriptor was `3`. Once it was closed, the same descriptor number could be reused for the second opening of `note.txt`.

4. Part 3. Count One Kind of Call

I restricted tracing to `openat`, saved the combined output, and used `grep` to count lines beginning with the system-call name. The `2>&1` redirection matters because `strace` normally writes its trace to standard error.

```
$ strace -e trace=openat ./syscalls 2>&1 | tee openat_trace.txt
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
openat(AT_FDCWD, "note.txt", O_RDONLY)  = 3
open returned descriptor 3
write sent 14 bytes
read got 14 bytes: hello, kernel
+++ exited with 0 +++
$ grep -c '^openat(' openat_trace.txt
4
```

Figure 7: Tracing and counting openat calls

Of the four observed `openat()` calls, two were for initialization files (the dynamic linker cache and the C library), and two were for `note.txt`. This shows why the number of system calls made by a program can exceed the number explicitly written in its source code.

5. Conclusion

This laboratory helped me understand how normal C file operations appear at the kernel interface. I observed that `ls` writes to file descriptor `1`, while my own program uses descriptor `3` to access a file. The recorded trace confirmed the 14-byte write and read, and the final experiment counted four `openat()` calls, including startup activity that was not explicitly part of the application logic. The experiment also showed how `strace` can help inspect program behavior without changing the program itself.